

Traffic Rule Violation Detection in Street Images

Harshal Rudra (MT2025706)

Manzil Baruah (MT2025715)

Sunil Joshi (MT2025726)

[GitHub](#) — [Models](#)

Abstract

This report presents a modular computer vision system for detecting traffic rule violations involving two-wheelers from single RGB street images. The system detects all motorcycles/scooters, identifies violations (more than two riders, riders not wearing helmets, or both), and extracts license plate numbers. The pipeline uses two fine-tuned YOLOv8n models for object detection, Hungarian matching for rider-to-vehicle association, and EasyOCR with preprocessing ensemble for license plate recognition. The total model size is ~ 105 MB, inference time is 17.3 ms per image (58 FPS on Apple M5), and detection mAP@50 reaches 0.970. The design emphasises robustness to occlusions, low lighting, and small plates while respecting strict resource constraints.

1 Introduction

Traffic rule enforcement on two-wheelers is critical for road safety, yet manual monitoring is resource-intensive. This project develops an automated computer vision system that, given a street image, detects two-wheelers, identifies helmet and rider-count violations, and reads license plates. The system must operate offline with a total model size under 250 MB and no large vision-language models. We present a lightweight, modular pipeline based on YOLOv8n and EasyOCR that achieves high detection accuracy and fast inference.

2 Problem Statement

Given a single RGB image captured from a street scene, the system must:

1. Detect all two-wheelers (motorcycles/scooters).
2. Identify whether any of the following violations occur:
 - More than two riders on a two-wheeler.
 - One or more riders not wearing a helmet.
 - Not wearing a helmet and more than two riders.
3. For each violating vehicle, extract and report the registration number (license plate).

3 Constraints

- **Model Size Limit:** Total size of all model files must not exceed 250 MB.
- **No Large Models:** Use of Vision-Language Models with $> 1\text{B}$ parameters is prohibited.
- **Offline Execution:** Internet access disabled during evaluation.
- **Inference Efficiency:** Reasonable processing time per image.

4 System Design and Implementation

4.1 Overall Pipeline

The system follows a multi-stage pipeline (Figure 1):

1. Object detection on the full image using a custom fine-tuned YOLOv8n model that detects four classes: ‘motorcycle’, ‘helmet’, ‘no_helmet’, and ‘license_plate’.
2. Rider detection using a separate YOLOv8n model pre-trained on COCO (person class only).
3. Association of detected persons to motorcycles using global Hungarian matching (each person assigned to at most one motorcycle; each motorcycle can have up to 3 rider slots).
4. Violation inference: rider count from assigned persons; helmet violation from presence of ‘helmet’ detections overlapping the rider’s head region (fallback to ‘no_helmet’ if person detector misses riders).
5. License plate cropping and OCR: for each violating motorcycle, select the best plate detection based on confidence and proximity, then apply preprocessing ensemble (original, 2x upscale, deskew, adaptive thresholding) and run EasyOCR.
6. Output JSON per image with list of violations.

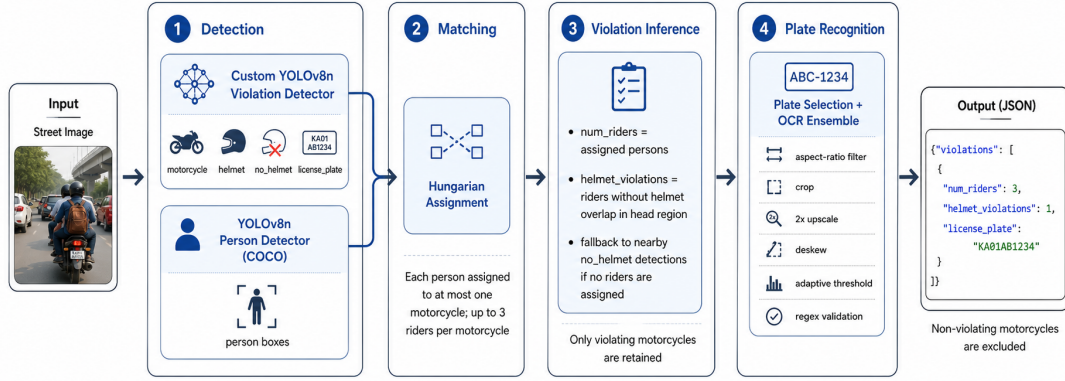


Figure 1: Overview of the proposed violation detection and license plate recognition pipeline.

Figure 1: System pipeline: two-detector design with Hungarian assignment and OCR ensemble.

4.2 Model Details

- **Violation Detector:** Fine-tuned YOLOv8n (6 MB) on a merged dataset of 11,479 images with 4 classes: ‘motorcycle’, ‘helmet’, ‘no_helmet’, ‘license_plate’.
- **Person Detector:** YOLOv8n (6 MB) with COCO weights, used without fine-tuning.
- **OCR:** EasyOCR (93 MB) with English character set.
- **Total model size:** 105 MB (well under 250 MB limit).

4.3 Dataset and Training

The final violation detector was trained on a merged multi-source Roboflow dataset. Sources included:

- Helmet & License Plate Detection
- Motorcycle License Plate Detection
- Traffic Violation Detection (retained only ‘No helmet’)
- With No Helmet
- Indian License Plate Detection

After filtering for required classes, the dataset contained 11,479 images (train: 9,758, validation: 1,721). The model was fine-tuned from ‘yolov8n.pt’ using Ultralytics default augmentations (mosaic early, random HSV, horizontal flip, translation, scale) with ‘close_mosaic=3’ in final epochs. Training used disk cache, automatic mixed precision, and cosine learning rate scheduler.

4.4 Inference Optimisations

Inference is performed on Apple M5 using MPS acceleration. The two detectors run sequentially; Hungarian matching adds negligible overhead. Multiple preprocessing variants for OCR (upsampling, deskewing, adaptive thresholding) are applied in parallel and the best recognised string (validated against Indian plate regex) is returned; otherwise an empty string is returned to avoid false positives.

5 Results

5.1 Detection Performance

The fine-tuned YOLOv8n achieved the following validation metrics on the 4-class test split:

Table 1: Per-class and overall detection metrics (validation set).

Class	Precision	Recall	mAP@50
helmet	0.926	0.948	0.970
no_helmet	0.909	0.895	0.954
motorcycle	0.907	0.931	0.969
license_plate	0.967	0.957	0.985
Overall	0.927	0.933	0.970

mAP@50:95 overall was 0.707.

5.2 Inference Speed

Average inference time per image: 17.3 ms (approximately 58 FPS) on Apple M5 using MPS. This satisfies the efficiency requirement.

5.3 Violation Classification and OCR Accuracy

The repository does not report separate end-to-end violation classification accuracy or exact-match OCR accuracy on a held-out evaluation set. However, the high detection precision/recall for ‘helmet’, ‘no_helmet’, and ‘motorcycle’ suggests strong violation detection capability. OCR is enhanced by preprocessing ensemble and regex filtering to avoid false positives.

6 Failure Case Analysis

Based on design choices and typical challenges, the following failure cases are anticipated:

- **Low lighting / night scenes:** Reduced detection recall for ‘no_helmet’ and license plates.
- **Occluded riders:** Person detector may miss heavily occluded riders; fallback to ‘no_helmet’ detections partially mitigates.

- **Small or blurry license plates:** OCR accuracy degrades; preprocessing helps but not sufficient for extreme blur.
- **Crowded scenes with multiple motorcycles:** Hungarian assignment reduces cross-assignment errors but may still cause misassignment when motorcycles overlap heavily.
- **Domain shift:** Training data mixes several sources but may not cover all traffic environments (e.g., different plate formats, helmet types).
- **Inconsistent annotations across merged datasets:** The ‘no_helmet’ class especially suffers from varying labelling standards.

6.1 Robustness Enhancements

The system incorporates several design choices to improve robustness:

- Two-detector design (custom + COCO person) improves rider localisation.
- Global Hungarian matching reduces double counting.
- Inverted helmet logic: relies on stronger ‘helmet’ detections rather than weak ‘no_helmet’.
- Fallback: if person detector misses riders, nearby ‘no_helmet’ boxes are used as rider proxies.
- Plate aspect-ratio filtering discards non-plausible boxes.
- OCR ensemble with four preprocessing paths.
- Regex validation against Indian plate format.
- Conservative abstention: empty string if no trustworthy OCR result.
- Training augmentations (mosaic, HSV, flips) improve generalisation.

7 Conclusion

We have implemented a resource-efficient computer vision system for detecting two-wheeler traffic violations. The pipeline uses two small YOLOv8n detectors (total 12 MB) plus EasyOCR (93 MB), well within the 250 MB limit. With detection mAP@50 of 0.970 and inference at 58 FPS, the system is both accurate and fast. Future work could improve low-light robustness (e.g., image enhancement preprocessing) and incorporate a more powerful but still lightweight OCR model.

8 Deliverables

The submission includes:

- `solution.py` implementing the `TrafficViolationDetector` class.
- `models/` directory with `violation_detector.pt`, `person_detector.pt`, and Easy-OCR weights.
- `requirements.txt` (dependencies: `ultralytics`, `easyocr`, `opencv-python`, `numpy`, `scipy`).
- `README.md` with setup and usage instructions.
- This report.